Technische Universität Bergakademie Freiberg
Institute of Numerical Mathematics and Optimization
Prof. Dr. O. Rheinbach
Dr. S. Köhler

Wintersemester 2026/27
21.10.2026

Praktikum Wissenschaftliches Rechnen / Scientific Programming Project

1. Exercise Sheet: Linear Regression in PYTHON

Exercise 1
These exercises guide you through implementing linear regression from scratch using NumPy.

Prerequisites:

- Python 3.8 or higher
- NumPy library
- Matplotlib library (for visualization exercises)
- Basic understanding of matrix operations

Background:

Linear regression models the relationship between input features X and target values y as:

$$y = w_0 + w_1 x_1 + w_2 x_2 + \ldots = X\mathbf{w} \tag{1}$$

Perfect linear data follows this relationship exactly without noise, making it ideal for testing implementations.
Remark: Linear regression assumes a linear relationship between features and target. It will fail on nonlinear relationships like polynomials, exponentials, or trigonometric functions. Testing on nonlinear data demonstrates the model's limitations.

1. Data Generation:

- Implement a function `create_perfect_linear_data` that generates synthetic data following a perfect linear relationship.
  - **Function name:** `create_perfect_linear_data`
  - **Parameters:**
    - `n_samples: int = 100` — Number of data points to generate
  - **Returns:**
    - `X: ndarray of shape (n_samples, 1)` — Input features
    - `y: ndarray of shape (n_samples,)` — Target values
  - **Implementation details:**
    - Use `np.random.default_rng(42)` for reproducibility
    - Generate X values randomly between 0 and 1
    - Use the relationship: $y = 2 + 3x$ (intercept=2, slope=3)
    - Return X as a 2D array with shape $(n, 1)$
- Implement a function `create_noisy_linear_data` that generates linear data with additive Gaussian noise.

  - **Function name:** `create_noisy_linear_data`
  - **Parameters:**
    - `n_samples: int = 100` — Number of data points
    - `noise_level: float = 2.0` — Standard deviation of Gaussian noise
  - **Returns:**
    - `X: ndarray of shape (n_samples, 1)` — Input features
    - `y: ndarray of shape (n_samples,)` — Noisy target values
  - **Implementation details:**
    - Use `np.random.default_rng(42)`
    - Generate X randomly between 0 and 1
    - True relationship: $y = 2 + 3x$
    - Add Gaussian noise: `rng.normal(0, noise_level, n_samples)`

- Implement a function `create_nonlinear_data` that generates data following a quadratic (nonlinear) relationship.
  - **Function name:** `create_nonlinear_data`
  - **Parameters:**
    - `n_samples: int = 100` — Number of data points
  - **Returns:**
    - `X: ndarray of shape (n_samples, 1)` — Input features
    - `y: ndarray of shape (n_samples,)` — Target values
  - **Implementation details:**
    - Use `np.random.default_rng(42)`
    - Generate X uniformly between 0 and 1
    - Relationship: $y = x^2$ (quadratic)
    - No noise (perfect quadratic curve)

2. Normal Equation Implementation: The normal equation provides an analytical solution to linear regression. Given data matrix $X \in \mathbb{R}^{n \times d}$ with full rank and targets $\mathbf{y} \in \mathbb{R}^n$, the optimal weights are:

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y} \tag{2}$$

To include an intercept term, we augment X with a column of ones: $\tilde{X} = [1 \mid X]$.
Implement a class `LinearRegressionNormalEquation` that solves linear regression using the normal equation.

- Class Specifications
  - **Attributes:**
    - `weights: ndarray | None` — Learned weights (initialized to `None`)
  - **Methods:**
    - `fit(X, y)` — Compute optimal weights
    - `predict(X)` — Make predictions on new data
- Method: `fit`
  - **Parameters:**

    - `X: ndarray of shape (n_samples, n_features)`
    - `y: ndarray of shape (n_samples,)`
  - **Implementation:**
    a) Augment X with intercept: `np.column_stack([np.ones(len(X)), X])`
    b) Compute X^TX using `@` operator
    c) Compute $(X^TX)^{-1}$ using `np.linalg.inv`
    d) Compute $\mathbf{w} = (X^TX)^{-1}X^T\mathbf{y}$
    e) Store in `self.weights`

- Method: `predict`
  - **Parameters:**
    - `X: ndarray of shape (n_samples, n_features)`
  - **Returns:**
    - `y_pred: ndarray of shape (n_samples,)`
  - **Implementation:**
    a) Augment X with intercept column
    b) Return $X\mathbf{w}$ using `@` operator

3. Gradient Descent Implementation: Gradient descent iteratively minimizes the loss function:

$$L(\mathbf{w}) = \frac{1}{2n}\sum_{i=1}^{n}(y_i - \mathbf{w}^T\mathbf{x}_i)^2 \tag{3}$$

The gradient is:

$$\nabla L(\mathbf{w}) = \frac{1}{n}X^T(X\mathbf{w} - \mathbf{y}) \tag{4}$$

Update rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\nabla L(\mathbf{w}^{(t)}) \tag{5}$$

Implement `LinearRegressionGradientDescent` using batch gradient descent.

- Class Specifications
  - **Constructor Parameters:**
    - `learning_rate: float = 0.01` — Step size α
    - `n_iterations: int = 1000` — Number of optimization steps
  - **Attributes:**
    - `learning_rate: float`
    - `n_iterations: int`
    - `weights: ndarray | None`
    - `loss_history: list[float]` — Track loss over iterations

- Method: `fit`
  - **Implementation:**
    a) Augment X with intercept column

b) Initialize weights to zeros

c) For each iteration:

* Compute predictions: $\hat{\mathbf{y}} = X\mathbf{w}$
* Compute error: $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$
* Compute gradient: $\nabla L = \frac{1}{n} X^T \mathbf{e}$
* Update: $\mathbf{w} := \mathbf{w} - \alpha \nabla L$
* Store loss: $\frac{1}{2}\text{mean}(\mathbf{e}^2)$

- Method: `predict`

  Same as for `LinearRegressionNormalEquation`.

4. Stochastic Gradient Descent Implementation: Mini-batch SGD updates weights using small random batches rather than the full dataset:

a) Shuffle data

b) Divide into batches of size m

c) For each batch: compute gradient and update

This balances computation speed with gradient stability.
Implement `LinearRegressionSGD` using mini-batch stochastic gradient descent.

- Class Specifications
  - **Constructor Parameters:**
    * `learning_rate: float = 0.01`
    * `n_iterations: int = 1000` — Number of epochs
    * `batch_size: int = 32`
  - **Attributes:**
    * `learning_rate: float`
    * `n_iterations: int`
    * `batch_size: int`
    * `weights: ndarray | None`
    * `loss_history: list[float]`
- Method: `fit`
  - **Implementation:**

    a) Augment X with intercept

    b) Initialize weights to zeros

    c) For each epoch:

    * Shuffle data: `np.random.permutation(n_samples)`
    * For each batch:
      · Extract batch using `start_idx:end_idx`
      · Compute batch predictions and error
      · Compute batch gradient
      · Update weights
    * Store average epoch loss

- Method: `predict`

  Same as for `LinearRegressionNormalEquation`.

5. R^2 Score Metric: The coefficient of determination (R^2) measures the proportion of variance in the target explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{6}$$

where $\bar{y}$ is the mean of target values.
Interpretation:

- $R^2 = 1$: Perfect predictions
- $R^2 = 0$: No better than predicting the mean
- $R^2 < 0$: Worse than predicting the mean

Implement a function `compute_r2_score` that calculates R^2.

- Specifications
  - **Function name:** `compute_r2_score`
  - **Parameters:**
    - `y_true:` `ndarray` — True target values
    - `y_pred:` `ndarray` — Predicted values
  - **Returns:**
    - `r2:` `float` — R^2 score
  - **Implementation:**
    a) Compute $SS_{\text{tot}} = \sum(y_i - \bar{y})^2$
    b) Compute $SS_{\text{res}} = \sum(y_i - \hat{y}_i)^2$
    c) Return $1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$

6. RMSE Metric: Root Mean Squared Error (RMSE) measures the average prediction error in the same units as the target:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{7}$$

Lower values indicate better predictions. Unlike R^2, RMSE is in the same units as y, making it more interpretable.
Implement a function `compute_rmse` that calculates RMSE.

- Specifications
  - **Function name:** `compute_rmse`
  - **Parameters:**
    - `y_true:` `ndarray` — True values
    - `y_pred:` `ndarray` — Predicted values
  - **Returns:**
    - `rmse:` `float` — Root mean squared error
  - **Implementation:**

a) Compute MSE: `np.mean((y_true - y_pred)**2)`

b) Return `np.sqrt(mse)`

7.: Comparison: Comparing different optimization methods on various datasets reveals their strengths and weaknesses. This exercise integrates all previous implementations into a comprehensive evaluation.
Implement a function `demonstrate_methods` that:

a) Creates three datasets (perfect linear, noisy linear, nonlinear)

b) Trains all three models on each dataset

c) Computes R^2 and RMSE for each combination

d) Creates a 3×3 grid of scatter plots with fitted lines

- Specifications
  - **Function name:** `demonstrate_methods`
  - **Parameters:** None
  - **Returns:** None (creates plots and prints results)
  - **Requirements:**
    - Use `matplotlib.pyplot` for plotting
    - Create figure with `plt.subplots(3, 3, figsize=(15, 12))`
    - Set overall title
    - For each subplot: scatter plot data, plot fitted line, show metrics
    - Print learned weights for perfect data
- Datasets to Create
  a) Perfect linear: 100 samples
  b) Noisy linear: 100 samples, noise_level=2.0
  c) Nonlinear: 100 samples
- Models to Test
  a) Normal Equation
  b) Gradient Descent: `learning_rate=0.1, n_iterations=500`
  c) SGD: `learning_rate=0.1, n_iterations=500, batch_size=32`
- Expected Output
  - 9 subplots showing model performance on different datasets
  - Console output showing learned weights (should be close to [2, 3])